SMARTCANTEEN

OOP CAPSTONE PROJECT

Understanding the User Journey, System Logic, and Java Code

Purpose	A structured guide to understanding the application from startup through checkout, kitchen preparation, and order tracking.
Focus	Java, Swing GUI, classes and objects, the four OOP pillars, and the main code flow.
Use	Study aid for understanding the project and preparing for a capstone demonstration or defense.

Prepared as a clean, organized study document. The explanations and examples preserve the structure and information of the guide.

How to Use This Guide

This guide follows the same order in which the application runs. Start with the customer journey, then study what happens behind the scenes, and finally connect the code to the OOP concepts and likely defense questions.

The four main parts are: (1) the user journey, (2) the system process, (3) the code and logic, and (4) capstone defense preparation.

1. First, understand what your capstone is

SmartCanteen is a self-service canteen ordering system for Cebu Eastern College. Customers order food from multiple canteen stalls, pay for their orders, and track their status while staff manage preparation and pickup.

Project detail	Description
Programming language	Java
GUI framework	Java Swing, which provides windows, buttons, text fields, tabs, and dialogs.
Design	The code separates the graphical interface (gui) from the business objects and logic (model).
Data storage	The application uses sample data and objects held in memory. No database connection is evident in the provided source.
Main purpose	Coordinate customer orders and payments with staff preparation and pickup tracking.

Important: the project demonstrates OOP and a working ordering workflow, but it should not be described as a database-backed system unless that functionality is added.

2. The complete user journey

Step	What the user/system does
1. Open SmartCanteen	Main.java launches the customer window.
2. Select a canteen stall	Browse the five stores and view their menus and prices.
3. Add food to the cart	Choose quantities. The system creates or updates a store-specific order.
4. Proceed to checkout	Select Exact Cash or Big Bill for each store and validate payment.
5. Process payment and submit orders	Calculate totals and change, assign queue numbers, and publish orders to the shared order board.
6. Display the receipt and tracker	Show the receipt, individual store queues, payment details, and order statuses.
7. Staff prepare and mark orders ready	The customer tracker updates when staff change the order status.

Step 1 — The program starts

The starting point is src/smartcanteen/Main.java. Its job is to launch the application and display the main window. It does not handle food prices or payment calculations.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(
```

```
                UIManager.getSystemLookAndFeelClassName()
            );
        } catch (Exception ignored) {
            // Use the default appearance if necessary
        }

        new MainFrame().setVisible(true);
    });
}
```

Understand each part:

• main() — the entry point where Java starts executing the program.
• SwingUtilities.invokeLater() — schedules GUI initialization on Swing's event-dispatch thread.
• UIManager.setLookAndFeel() — attempts to make the interface look consistent with the operating system.
• new MainFrame() — creates an object representing the main application window.
• setVisible(true) — makes that window appear.

Defense explanation: "The program begins in the Main class. It initializes the Java Swing interface and creates the MainFrame object, which serves as the main window for customer ordering."

Step 2 — The main window prepares the system

Open gui/MainFrame.java. Inside its constructor, the program creates the objects that the rest of the application needs.

```
this.stores = SampleData.buildStores();

this.customer = new Customer(
    "C-0001", "Guest Customer", "N/A"
);

this.cart = new LinkedHashMap<>();

this.orderBoard = new OrderBoard();
```

Code	Purpose
SampleData.buildStores()	Creates five canteen stalls and their sample menus.
new Customer(...)	Creates the initial customer object.
new LinkedHashMap<>()	Creates a cart that associates each store with its order.
new OrderBoard()	Creates the shared place where submitted orders are published for staff and customer tracking.

The main window also creates a StorePanel for every store and a CartPanel for the shopping cart. Creating an object and displaying it are different things: new MainFrame() constructs the window, while setVisible(true) displays it.

Step 3 — The customer browses the menu

Menu information originates in model/SampleData.java. The system creates five example stores:

Example store 1	Example store 2	Example store 3	Example store 4	Example store 5
TUDTUD STORE	SUAREZ STORE	SHINJI STORE	MARIEGINE STORE	PRINCESS STORE

StorePanel.java displays a store's menu. When the customer clicks a store tab, MainFrame.selectStoreTab() switches to that store's panel. When the customer clicks Add to Cart, the button's event handler calls MainFrame.addToCart().

```
public void addToCart(Store store, MenuItem item, int qty) {
    SubOrder subOrder = cart.get(store);

    if (subOrder == null) {
```

```
            subOrder = new SubOrder(
                "SO-" + (++subOrderCounter), store
            );
            cart.put(store, subOrder);
        }

        subOrder.addItem(new OrderItem(item, qty));
        cartPanel.refresh();
    }
```

Line-by-line logic:

1. cart.get(store) checks whether the customer already has an order for that stall.
2. if (subOrder == null) checks whether one needs to be created.
3. new SubOrder(...) creates the store-specific order if necessary.
4. cart.put(store, subOrder) associates that order with its store.
5. new OrderItem(item, qty) creates an object representing the chosen item and quantity.
6. subOrder.addItem(...) adds the item to the order.
7. cartPanel.refresh() refreshes the cart display.

Why separate sub-orders? If the customer buys rice from TUDTUD STORE and milk tea from MARIEGINE STORE, the system keeps those purchases in separate sub-orders rather than treating them as one kitchen order.

Step 4 — The system calculates the order total

The cart uses three important model classes: MenuItem represents a menu item; OrderItem represents a menu item together with its quantity; SubOrder represents all items ordered from one specific stall.

Item	Quantity	Unit price	Line total
Steamed Rice	2	₱15	₱30
Chicken Adobo	1	₱65	₱65
Store subtotal			₱95

The program calculates each line total using OrderItem.getLineTotal(), then SubOrder.recalculateSubTotal() adds those line totals together.

```
private void recalculateSubTotal() {
    subTotal = 0;

    for (OrderItem oi : itemsList) {
        subTotal += oi.getLineTotal();
    }
}
```

Read it as: start at zero, visit every item in the order, and add its line total to the subtotal. This method is called whenever an item is added to the sub-order.

Step 5 — The customer checks out and pays

Open gui/CheckoutDialog.java. When the customer clicks the checkout button, the system opens the checkout dialog. The customer selects a payment method for each store represented in the cart.

Payment type	How it works
Exact Cash	The amount tendered is set to the store subtotal.
Big Bill	The customer enters one or more bills, and the system calculates the combined amount.

For Big Bill, the code checks that the amount is present, valid, positive, and sufficient to pay for the order. If validation fails, it displays an error and stops checkout.

Payment example	Amount tendered	Result
Exact Cash	₱95	₱0 change
Big Bill	₱100	₱5 change
Insufficient payment	₱50	Not accepted

The change calculation in SubOrder.java is:

```
public double calculateStoreChange() {
    changeToReturn = cashTendered - subTotal;
    return changeToReturn;
}
```

After payment inputs have been checked, CheckoutDialog.onConfirm() creates a PaymentProcessor and adds the sub-orders to it. processCheckout() calculates change for each store, adds each subtotal to the grand total, adds each change amount to the overall change total, marks each sub-order PAID, and creates change-preparation alerts for Big Bill payments.

Important distinction: the system simulates recording and processing a cash payment. It does not connect to a bank, payment gateway, or actual electronic payment service.

Step 6 — The system publishes the order to the kitchen

Once checkout succeeds, the system submits every paid sub-order to the shared OrderBoard. Think of OrderBoard as a shared noticeboard inside the running application.

```
for (SubOrder so : processor.getSubOrdersList()) {
    parent.getOrderBoard().submit(so);
}
```

The OrderBoard.submit() method adds an order to a list and calls notifyListeners(). The kitchen display and customer tracker register listeners that refresh their interfaces when the board changes. This is an example of the Observer design pattern, or an observer-style publish-and-subscribe mechanism.

In simpler terms, instead of forcing the customer window to manually refresh the kitchen window, the shared board tells registered listeners when something changes. Both windows can respond to the same underlying order object while the application is running.

Step 7 — The staff prepare the food

The staff interface is implemented by KitchenDisplayFrame.java, which creates the staff window and tabs for stores, and KitchenStorePanel.java, which displays orders for a particular store and provides status buttons.

Status	Meaning
PENDING	The order has been created but not paid.
PAID	Checkout has been processed successfully.
PREPARING	Staff have started preparing the order.
READY	The order is ready for pickup.
COMPLETED	A final state is defined in the model, although the current staff controls shown in the code advance orders to PREPARING and READY.

After payment, the customer sees the receipt and queue number. Staff can then click Start Preparing Order, followed by Mark Ready for Pickup.

```
private void advance(
        SubOrder subOrder, SubOrder.Status next) {

    subOrder.updateStatus(next);
    board.touch();
}
```

What happens:
1. The method receives the selected order and its next status.
2. updateStatus(next) changes the order's status.
3. board.touch() notifies the board's listeners.
4. The kitchen display and customer tracker refresh to show the new status.

The system tracks the order as ready for pickup; it does not automatically verify that the customer has physically collected it.

Step 8 — The receipt is generated and the cart is cleared

In CheckoutDialog.java, after successful processing, the program creates a MasterReceipt:

```
MasterReceipt receipt = new MasterReceipt(
    "R-" + (System.currentTimeMillis() % 100000),
    parent.getCustomer().getName(),
    processor
);
```

This passes the generated receipt identifier, customer name, and payment processor into the receipt object. MasterReceipt.generateSummary() builds the receipt text, including:

- Customer name and receipt number.
- Each store's name and queue number.
- Ordered items and their line totals.
- Each store's subtotal.
- Cash paid and change.
- Overall grand total.

The checkout dialog then closes, the cart is cleared, and the receipt/tracker window opens. The workflow is complete from the application's perspective: payment has been processed, orders have been published to the kitchen, and the

customer can track their progress.

3. Now let's understand the OOP concepts in your actual code

Your instructor may ask, “Where is the OOP in your project?” You need to point to specific classes and explain why they demonstrate each concept. The four major OOP pillars are encapsulation, inheritance, polymorphism, and abstraction.

A. Encapsulation — protecting and organizing data

Encapsulation means combining data and the methods that work with that data inside a class, while controlling how that data is accessed. Look at model/SubOrder.java:

```
public class SubOrder {
    private String subOrderId;
    private double subTotal;
    private double cashTendered;
    private Status status;
}
```

The fields are private, which means other classes cannot directly access them by their field names. Instead, the class provides methods such as:

```
public double getSubTotal() {
    return subTotal;
}

public void setCashTendered(double cashTendered) {
    this.cashTendered = cashTendered;
}
```

For example, CheckoutDialog calls setCashTendered() rather than directly assigning a value to the field.

Defense answer: “We implement encapsulation by keeping the attributes of classes private and providing public methods to access or update them. This organizes the data and controls how other parts of the program interact with it.”

A useful nuance: encapsulation does not automatically guarantee that every value is valid. The setter shown above does not itself reject negative amounts; payment validation is handled elsewhere.

B. Inheritance — reusing a parent class

The inheritance relationship is written using extends. MenuItem is an abstract parent class, and FoodItem, BeverageItem, and ComboMeal are subclasses.

```
public class FoodItem extends MenuItem {
    // Food-specific behavior
}
```

Class	Role
MenuItem	Abstract parent; common menu-item information such as name, ID, base price, and store.
FoodItem	Food-specific behavior, including topping fees.
BeverageItem	Drink-specific behavior, including size fees.
ComboMeal	Bundled items and discount behavior.

This means FoodItem inherits accessible behavior from MenuItem and can define its own additional attributes and methods. The parent class contains common information, so subclasses do not need to recreate all common functionality.

Defense answer: “Inheritance is demonstrated by FoodItem, BeverageItem, and ComboMeal extending the abstract MenuItem class. This allows the subclasses to reuse common menu-item functionality while implementing their own specialized behavior.”

C. Polymorphism — one method, different behavior

The parent class declares methods such as:

```
public abstract double calculateTotalPrice();

public abstract String getItemDetails();
```

Each subclass implements those methods differently.

Class	How its price is calculated
FoodItem	Base price plus any topping fees.
BeverageItem	Base price plus the fee for its selected size.
ComboMeal	Component prices plus the base price, minus the discount, with a minimum of zero.

```
item.calculateTotalPrice();
```

The application does not need a separate call for every specific subclass. Java invokes the overridden implementation belonging to the actual object. That is polymorphism.

Defense answer: “Polymorphism is demonstrated when different subclasses override calculateTotalPrice() and getItemDetails(). The same method call can produce different results depending on whether the object represents food, a beverage, or a combo meal.”

D. Abstraction — hiding implementation details

A useful example is model/PaymentProcessor.java. The checkout interface does not need to perform every calculation itself. Instead, it invokes:

```
processor.processCheckout();
```

The PaymentProcessor class handles the calculations and updates payment-related information. The caller can use the method without needing to know every internal step.

There are two related ideas here:

• **Abstract class:** MenuItem is an abstract class that defines behavior subclasses must implement.
• **Abstraction as an OOP principle:** PaymentProcessor exposes a higher-level operation, processCheckout(), while hiding detailed processing logic behind it.

Defense answer: “Abstraction is demonstrated by the abstract MenuItem class and the PaymentProcessor class. The parent class defines common behavior that subclasses must implement, while PaymentProcessor provides a single method for processing checkout without exposing every internal calculation to the GUI.”

4. The other classes you need to understand

Use this code map when navigating the project in IntelliJ IDEA or another Java IDE.

File / class	Purpose
Main.java	Starts the program and opens MainFrame.
gui/MainFrame.java	Coordinates the customer window, store selection, cart, and shared OrderBoard.
gui/StorePanel.java	Displays one store’s menu and handles Add to Cart actions.
gui/CartPanel.java	Displays cart items, subtotals, grand total, and checkout controls.
gui/CheckoutDialog.java	Collects payment choices, validates amounts, and starts checkout.
model/Store.java	Represents a canteen stall and holds its menu.
model/MenuItem.java	Defines the shared menu-item structure.
model/FoodItem.java	Handles food-specific properties and topping prices.
model/BeverageItem.java	Handles drink sizes, ice level, and upsize fees.
model/ComboMeal.java	Combines menu items and applies a discount.
model/OrderItem.java	Associates a menu item with its quantity and line total.
model/SubOrder.java	Stores one stall’s items, subtotal, payment information, queue number, and status.
model/PaymentProcessor.java	Validates payment sufficiency and calculates totals and change.
model/OrderBoard.java	Shares submitted orders and notifies registered listeners.
model/MasterReceipt.java	Builds the receipt summary.
gui/ReceiptDialog.java	Displays the receipt and live order tracker.
gui/KitchenDisplayFrame.java	Creates the staff window and store tabs.
gui/KitchenStorePanel.java	Shows store orders and allows staff to advance order statuses.
model/SampleData.java	Creates the five sample stalls and their menus.

5. Follow one complete example

A customer named Alex orders two steamed rice servings and one chicken adobo from TUDTUD STORE.

Item	Quantity	Unit price	Line total
Steamed Rice	2	■15.00	■30.00
Chicken Adobo	1	■65.00	■65.00
Subtotal			■95.00

Connect that example to the real source code:

1. SampleData.java provides the food objects and prices.
2. StorePanel.java lets the customer choose quantities and add items.

3. MainFrame.addToCart() creates or retrieves the store's SubOrder.
4. OrderItem.java calculates the line totals.
5. SubOrder.java recalculates the store subtotal.
6. CheckoutDialog.java collects and validates payment.
7. PaymentProcessor.java calculates change and the grand total.
8. OrderBoard.java publishes the paid order to the staff interface.
9. MasterReceipt.java generates the receipt summary.
10. KitchenStorePanel.java lets staff update the order status.

If you can explain those ten steps in your own words, you already understand a large part of the application's architecture.

6. Important limitations to know before your defense

Understanding what the code does also means knowing what it does not do.

Limitation	What to say accurately
No persistent database is evident	Menus, cart, and submitted orders are held in memory. Closing the program loses that runtime data.
Payment is simulated	It calculates cash tendered and change; it does not process real electronic payments.
Some menu options may not be exposed in the interface	BeverageItem supports size and ice-level changes, but the displayed menu controls should be checked before claiming customers can customize them through the GUI.
COMPLETED status exists but is not shown in current staff controls	The current controls shown advance orders to PREPARING and READY. Do not claim the full pickup-completion workflow is implemented unless verified or added.
Queue numbers are generated in memory	They are not persistent queue numbers saved in a database.

These are not reasons to dismiss the project. They are boundaries of the current implementation that you should understand and explain honestly.

7. Practice your capstone defense

Try answering these questions without looking at the explanations above. Suggested answers are provided for study.

1. What happens when the customer clicks Add to Cart?

The StorePanel event handler calls MainFrame.addToCart(). The main window finds or creates a SubOrder for the selected store, creates an OrderItem with the chosen quantity, adds it to the sub-order, and refreshes the cart.

2. Why does the project use a SubOrder for each store?

Because one customer can order from multiple stalls. Each SubOrder keeps that store’s items, subtotal, payment information, queue number, and status separate.

3. What is the difference between inheritance and polymorphism here?

Inheritance allows FoodItem, BeverageItem, and ComboMeal to reuse the MenuItem structure. Polymorphism allows those subclasses to implement calculateTotalPrice() differently while the application uses the common method.

4. How does the kitchen display receive new orders?

CheckoutDialog submits paid SubOrder objects to OrderBoard. The board stores the orders and notifies registered listeners, allowing the staff display to refresh.

5. Does the system use a database?

The provided implementation uses SampleData and in-memory collections. There is no evident database connection in the supplied source, so order data is not persistently stored.

6. What is the purpose of PaymentProcessor?

It handles payment-processing logic, including checking that tendered cash covers each subtotal, calculating change, accumulating the grand total, marking orders as paid, and generating change-preparation alerts.

7. What happens if the customer enters too little cash?

CheckoutDialog checks the entered amount against the relevant SubOrder subtotal. If it is insufficient, an error message appears and checkout stops before processing the payment.

8. What OOP concept does MenuItem demonstrate?

MenuItem is an abstract superclass. It holds common menu-item data and declares abstract methods that its subclasses must implement, supporting abstraction and inheritance.

Recommended next study step

The best way to learn the code is to examine it in the same order the application runs, rather than trying to memorize all the Java files at once.

Start with Main.java, then MainFrame.java, then StorePanel.java. After that, move to OrderItem.java and SubOrder.java, because those classes explain how a customer's selection becomes an actual order. Once you understand those, checkout and the OOP concepts will make much more sense.

End of study guide